Situational Soft Skills for IT Professionals

By Vinodh Kotalawala

Ice Breaker

Situation : 2 AM. Your phone buzzes. Production is down. Customers can't access the system. Your manager is calling. Social media is exploding.

What soft skill saves you?

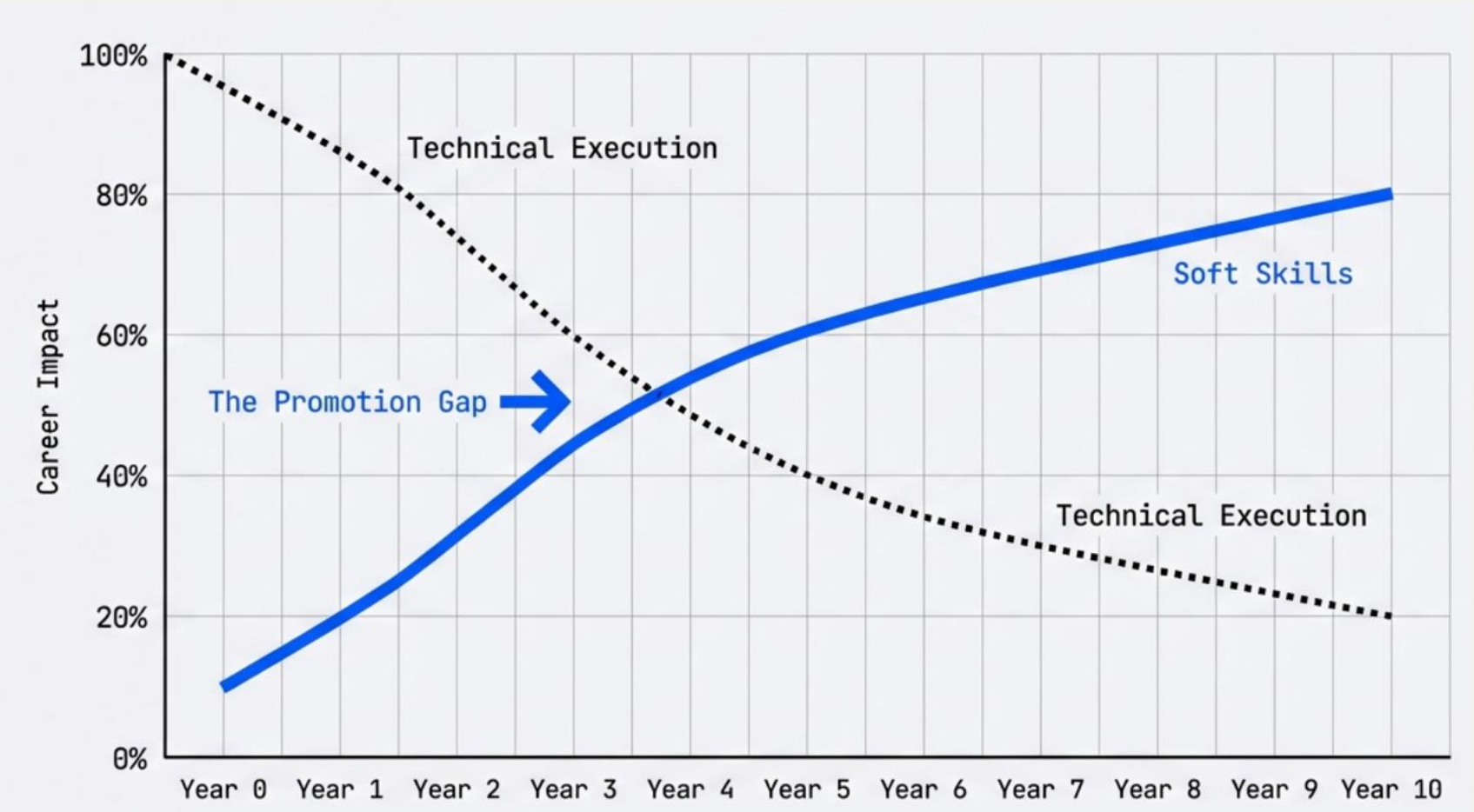
A) Leadership

B) Stress management

C) Flexibility

D) Conflict Management

The Truth



- Technical skills get you the interview
- Soft skills get you promoted
- By Year 10, soft skills are you job

The Traditional Approach

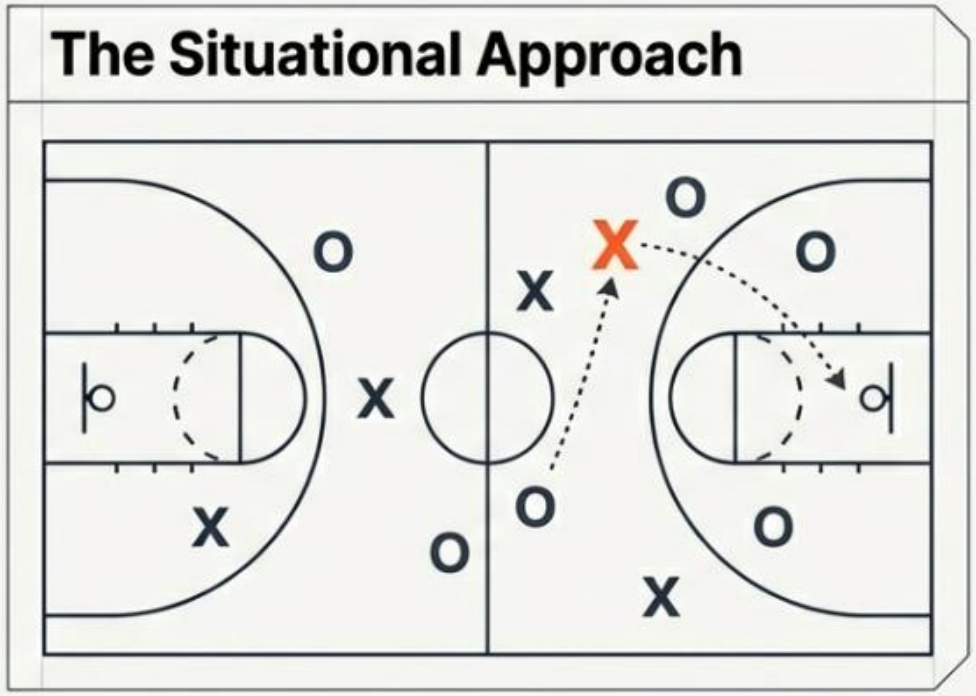
WHAT YOU USUALLY HEAR ABOUT SOFT SKILLS:

1. Leadership
2. Communication
3. Interpersonal Skills
4. Work Ethic
5. Teamwork
6. Problem Solving
7. Flexibility
8. Conflict Management

The advice: "Be good at all of them"

The problem: Doesn't tell you WHICH to use WHEN

The Framework- Situational Dominance



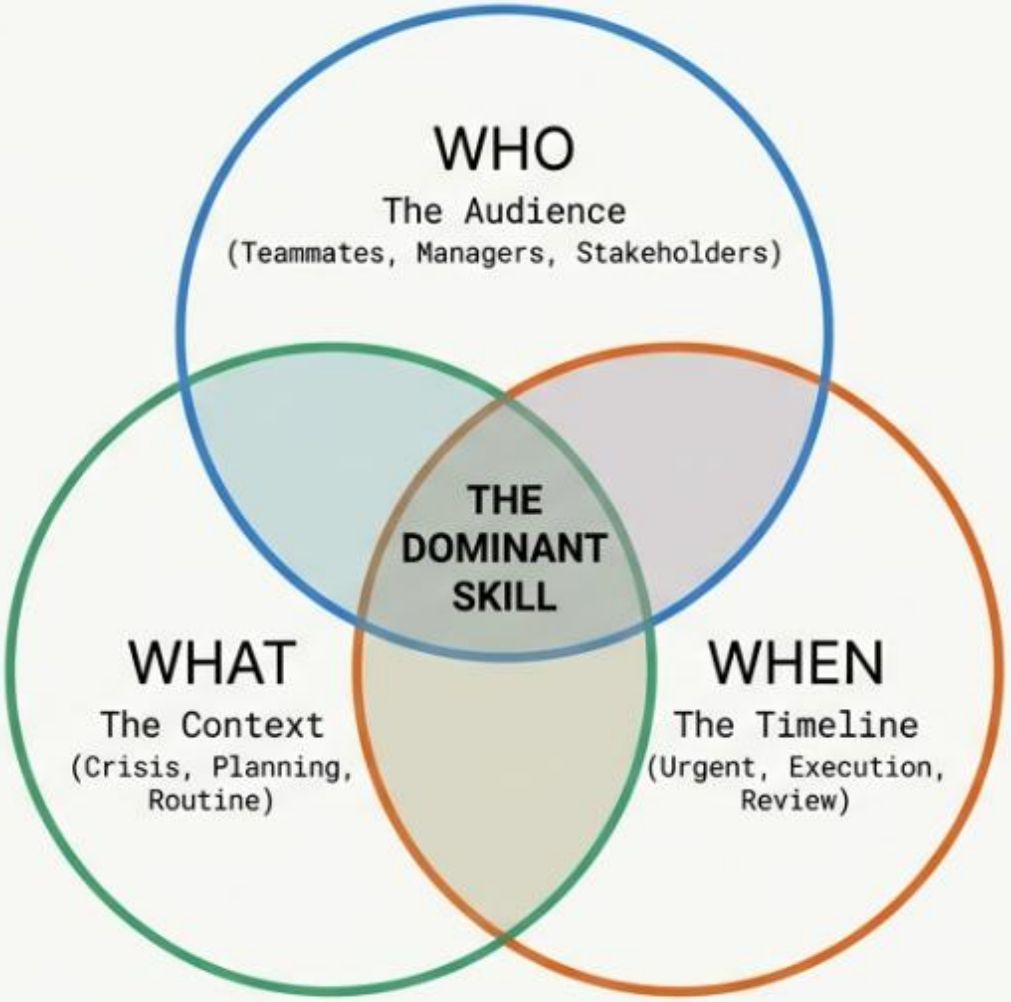
Concept: Situational Dominance

Analogy: Everyone is on the court, but the context determines who takes the shot. In any specific moment, only ONE skill determines success.

EXAMPLES

- Crisis Situation- Communication
- Tight Deadline- Time Management
- Team Conflict – Emotional Intelligence

The Three Variables



The Who, What, When Breakdown

Context determines the required skill. True competence is the ability to adapt across three specific dimensions.

The Relational Dimension
WHO determines the skill?

Teammates	→	Communication, Teamwork
Managers	→	Accountability, Initiative
Stakeholders	→	Negotiation, Presentation
Users	→	Empathy, Problem-solving

The Environmental Dimension
WHAT determines the skill?

Crisis	→	Stress Management
Planning	→	Analytical Thinking
Innovation	→	Adaptability
Routine	→	Time Management

The Temporal Dimension
WHEN determines the skill?

Urgent	→	Quick Decision-Making
Planning	→	Strategic Thinking
Execution	→	Focus, Accountability
Review	→	Openness to Feedback

Scenario 1- Behind The Schedule

Situation :

- You estimated 3 days to complete a task , but now actual time 6 is days,
- Status: Still not done
- Reason - API issues you didn't expect
- Standup starts in 5 minutes
- Team lead will ask for an update
- Everyone's listening.

Context: You estimated 3 days. It has been 6. You aren't done due to unexpected API issues.

The Moment: Standup starts in 5 minutes. The Team Lead asks for an update.

Scenario 1- Behind The Schedule- Answer

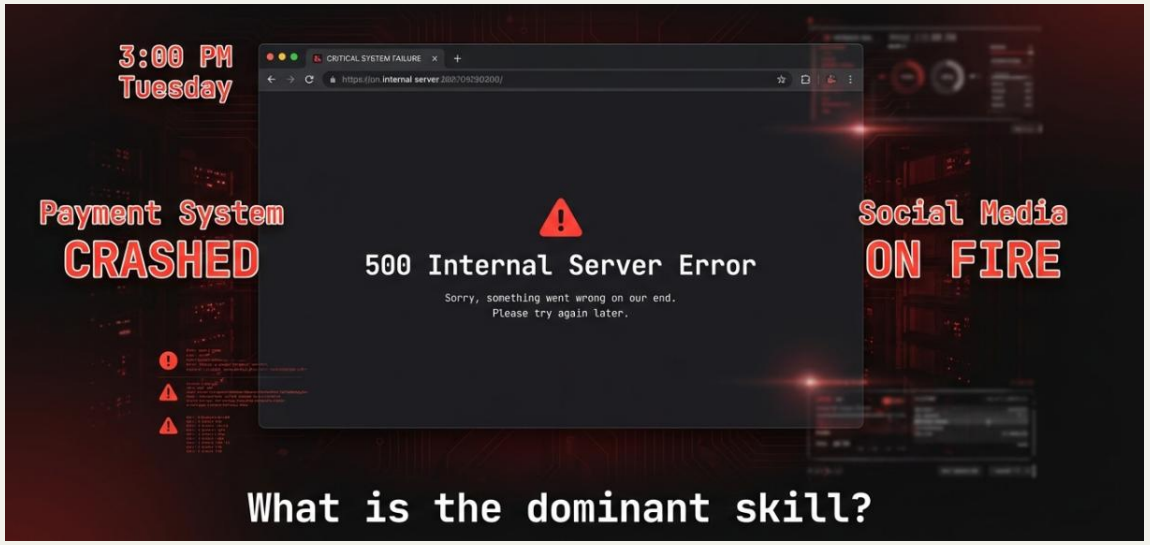
Answer	Accountability + Communication
Who	Team Lead
What	Routine work, behind schedule
When	Execution Phase
Dominant & Support	Accountability + Communication

Best Response
'I am behind by 3 days due to API issues I didn't anticipate. I've identified the blocker and need backend team's help. New realistic ETA: Friday."
Bad Responses
❌ Almost done" (lying) ❌ "It's harder than I thought" (vague excuse) ❌ *Stays silent* (avoiding)
Key : Own It, Communicate clearly. Offer Solution.

Scenario 2- Production Crisis

Situation :

3 PM Tuesday Payment system crashed . Users can't buy anything and social media exploding with complaints . Your manager calling and Business team panicking and you deployed the last update this morning.



Scenario 2- Production Crisis - Answer

Answer	**Stress Management + Communication**
Who	Manager + Business Stakeholders
What	Crisis, Emergency
When	Urgent, firefighting mode
Dominant and Support	Stress Management + Communication

Best Practices	**Bad Practices**
- Stay Calm (panic makes it worse) - Acknowledge immediately: "I see it, investigating" - Provide an update every 15 minutes - Debug systematically, not randomly - Ask for help if needed	- Panic and freeze - Blame QE and others - Go silent while trying to fix - Make random code changes
Key : How you handle the pressure , What they will remember	

Scenario 3 – Code Review

Situation :

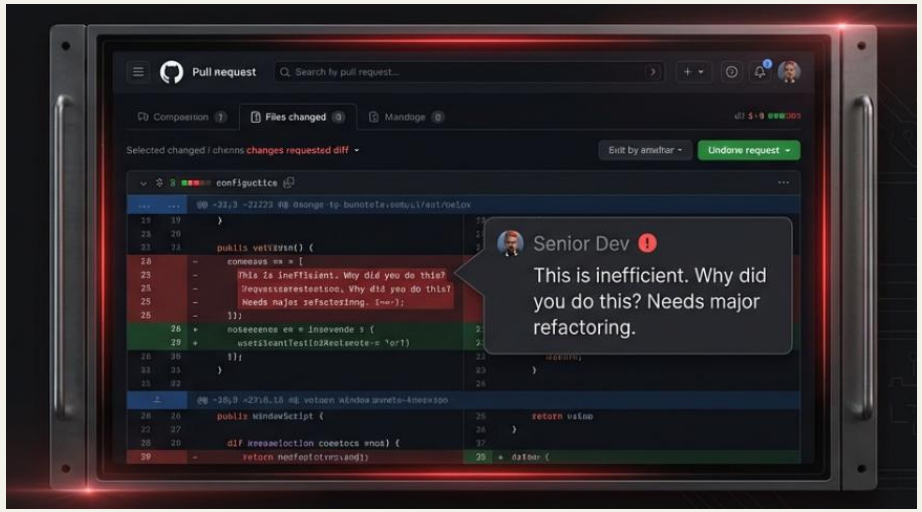
You submitted code. You worked hard for 2 days.

Senior dev reviews it and he is responding with following :

"This is inefficient" , "Why did you do it this way?"

"Violates our patterns“ "Needs major refactoring"

Your thoughts: My approach is fine. They're being harsh. They don't understand my reasoning.



Scenario 3 – Code Review - Answer

Answer	**Conflict Resolution + Emotional Intelligence**
Who	Senior Developer
What	Technical disagreement, Collaboration
When	Review
Dominant and Support	Conflict Resolution+ Emotional Intelligence

Best Response	**Bad Practices**
- Thanks for the review, can you help me understand the efficiency concerns? I chose this approach [provide your reason]. Is there a pattern I'm missing?	❌ "This works fine, what is the problem?" (defensive) ❌ "I'll change it " * without understanding (not learning) ❌ * Arguing in comments * (escalating conflicts)

Key : Code reviews = learning opportunities. Don't waste them.

Scenario 4 – Sprint Overload

Situation :

Product Owner wants: 20 story points "All high priority" , "CEO needs these for next release"

Your team's reality: Historical velocity: 12 points, One person on leave next week, Tech debt piling up, Realistic capacity: 10 points, Product owner expect a favorable answer

Scenario 4- Sprint Overload - Answer

Answer	**Negotiation + Analytical Thinking**
Who	Stakeholder (Product owner with power)
What	Setting expectation
When	Planning
Dominant and Support	Negotiation + Analytical Thinking

Best Response	**Bad Practices**
- Looking at our last 4 sprints, average velocity is 12 points. With one person on leave and critical tech debt, we will be able to commit 10 points this sprint. Let's prioritize: which 10 points deliver most value. We will discuss the rest for next sprint"	❌ "Okay, we will try all 20" (setting up failure) ❌ "That's too much" (complaining without data) ❌ * Silently accepting * (ensuring burnout)
Key : Negotiate with Data, not hope.	

Scenario 5– The Client Demo Disaster

Situation :

Client demo scheduled: 1 hour from now 30 stakeholders watching You're demoing the new dashboard feature

Status:

✓ Main charts work perfectly

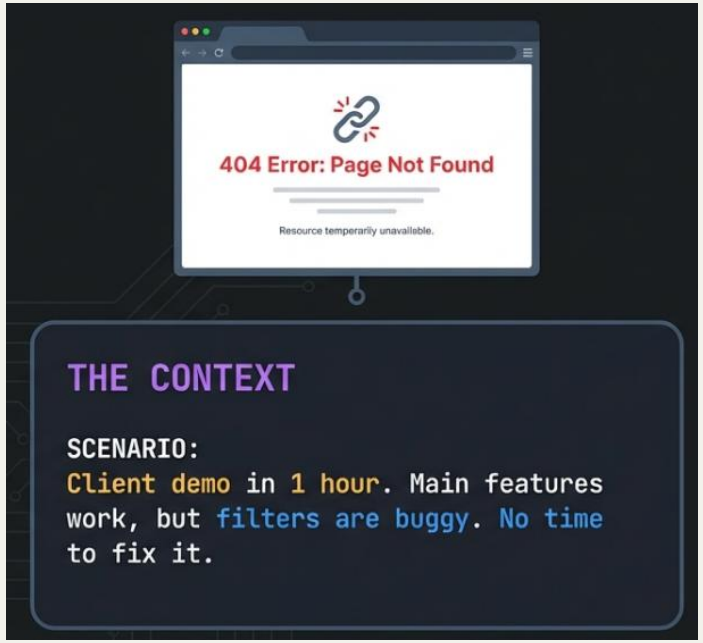
✓ Data loading works

✗ Filter function has bugs

✗ Export feature crashes sometimes

✗ Mobile view completely broken Client expects to see everything working.

No time to fix bugs.



Scenario 5- The Client Demo Disaster - Answer

Answer	**Presentation + Adaptability**
Who	Stakeholder (30 people, high visibility)
What	Demonstration
When	High stakes moment
Dominant and Support	Presentation + Adaptability

Best Response	**Bad Response**
- First demo what works flawlessly - Set expectations early : “Filters in final testing” - Focus on Value and Impact, not features - Have a backup (pre-recorded video/slides for broken parts) - Acknowledge in-progress work honestly	❌ Show buggy feature while apologizing throughout ❌ Avoid broken features without explaining why ❌ *Everything’s ready” *then it crashes ❌ Panic visibility when something breaks
Key : Clients don’t expect perfection. They expect Honesty and Value	

Common Mistakes

Using same approach in every situation

- Talking to teammates same way as executives
- Handling crisis same way as planning meeting
- Read WHO/WHAT/WHEN first, then adapt

Taking all feedback personally

- Code review = about CODE, not you
- Manager criticism = about WORK, not your worth
- Fix: Separate work from your identity
- Reality: Take it personal → Can't learn or grow

Over-apologizing for normal work activities

- Stop saying: - "Sorry to bother you" , "Sorry for the long email" ,
- Fix: Only apologize when you are wrong
- Truth: Excessive apologies kill your credibility

No boundaries, saying yes to everything

- Consequences:
  - Working on 5 projects at once, Responding to chats at midnight , Constantly dumping work
- Say instead: - "I'm at capacity, can take this next week" , "Need to prioritize – which is urgent?"
- Truth: Boundaries = Professional, not selfish

Never admitting "I don't know"

- Pretending makes you look worse, not better
- "I don't know, let me find out"
- " I haven't worked with that before let me try out"
- Truth: People respect honesty, not fake confidence

Trying to make everyone happy

- Reality: Impossible. Exhausting. Not effective
- Sometimes you must: - Push back on stakeholders , Disagree with your team ,
- Block a bad release
- Fix: Do what's RIGHT, not what makes everyone like you

Avoiding all disagreements and conflict

- You become a doormat
- Never push back on bad ideas
- Problems will not get solved
- Learn to disagree productively

Waiting for approval to show initiative

- Don't wait: - Don't ask if you can improve - just improve
- Don't wait to be told to help - help
- Don't wait for permission to lead – lead
- Remember: Initiative = Acting without being told
- Waiting for permission = Just following orders

How to Actually Improve

THE TRUTH ABOUT IMPROVEMENT:

❌ Reading about soft skills won't make you better ❌ Attending seminars won't make you better ❌ Knowing the theory won't make you better

- REPETITION - Use these skills daily. Not when you remember. Not when it's convenient. Every single day.
- REFLECTION - Learn from each interaction. What worked? What didn't? Why did that conflict escalate? Why did that explanation land perfectly? Spend 5 minutes at the end of each day reviewing your interactions.
- REAL PRACTICE
  - Very conversation is training. Every email you write is practice. Every meeting you join is a repetition. Every conflict that makes you uncomfortable? That's a learning opportunity. Every mistake that makes you cringe? That's data.

The Recap

- REMEMBER: There's NO single "best" soft skill.
- The best skill is the one that DOMINATES the situation you're in.
- Your job:
  - 1. READ the situation (WHO, WHAT, WHEN)
  - 2. IDENTIFY which skill dominates
  - 3. APPLY that skill effectively
  - 4. SUPPORT with other skills

Thank You!